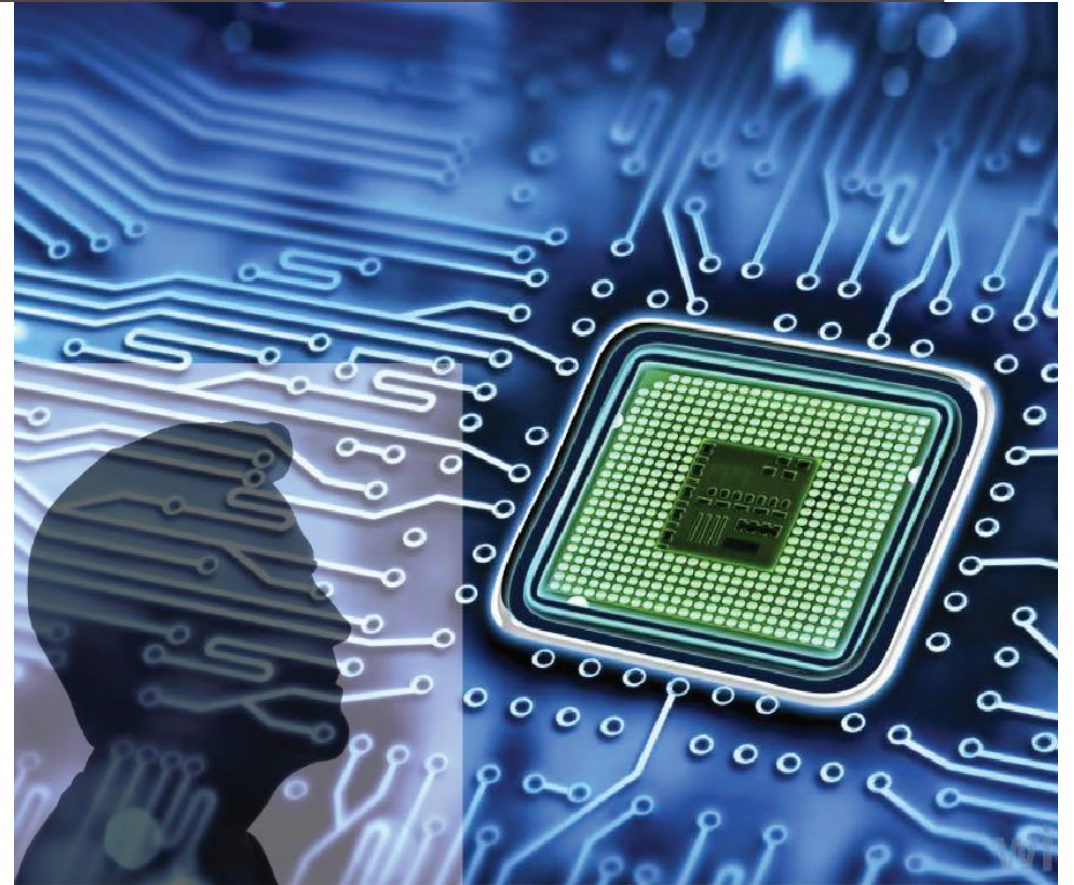


VLSI DESIGN FLOW

Week-10 Lecture-2 STA

Sneh Saurabh
23rd October, 2024

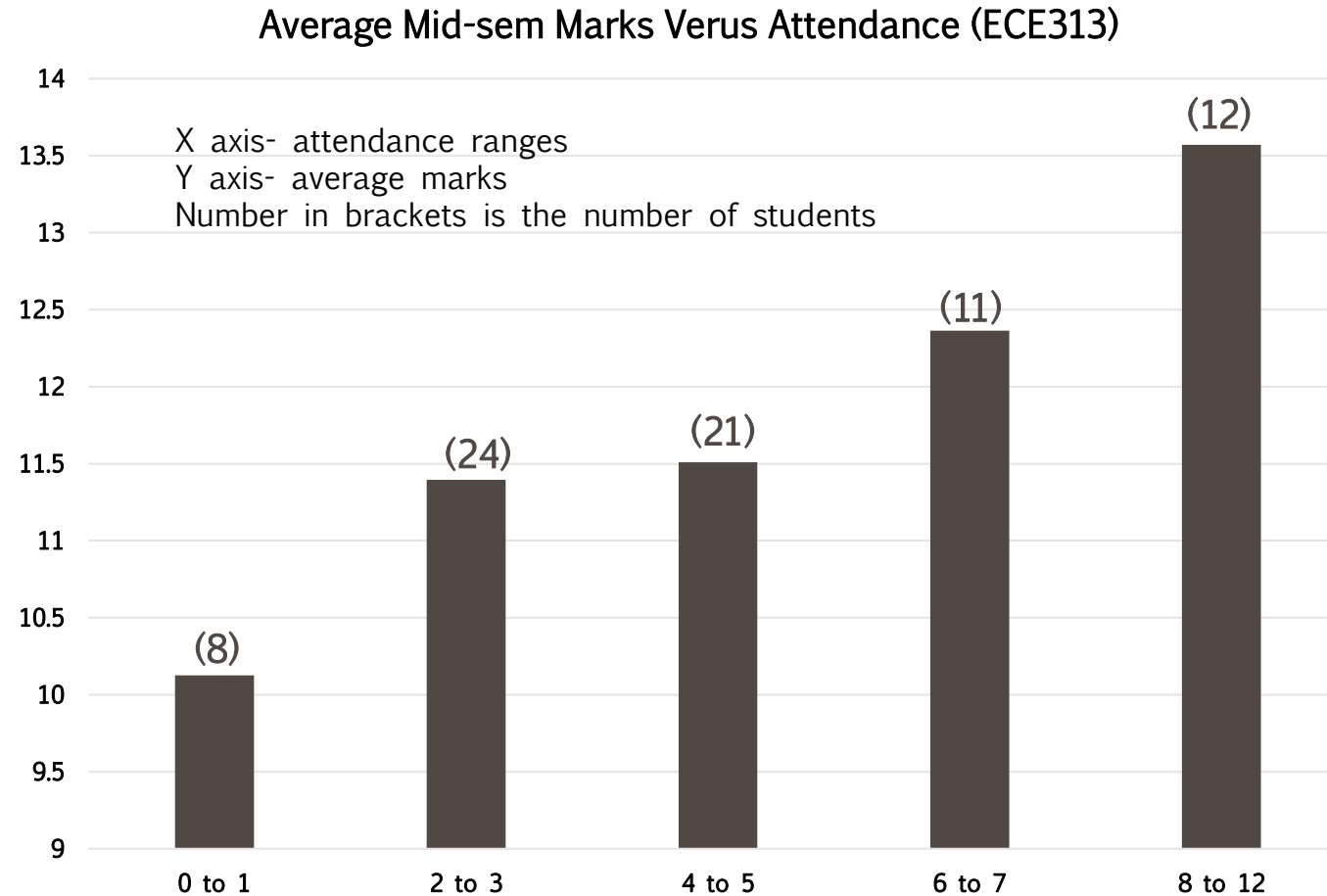


VDF: Revision/Announcements

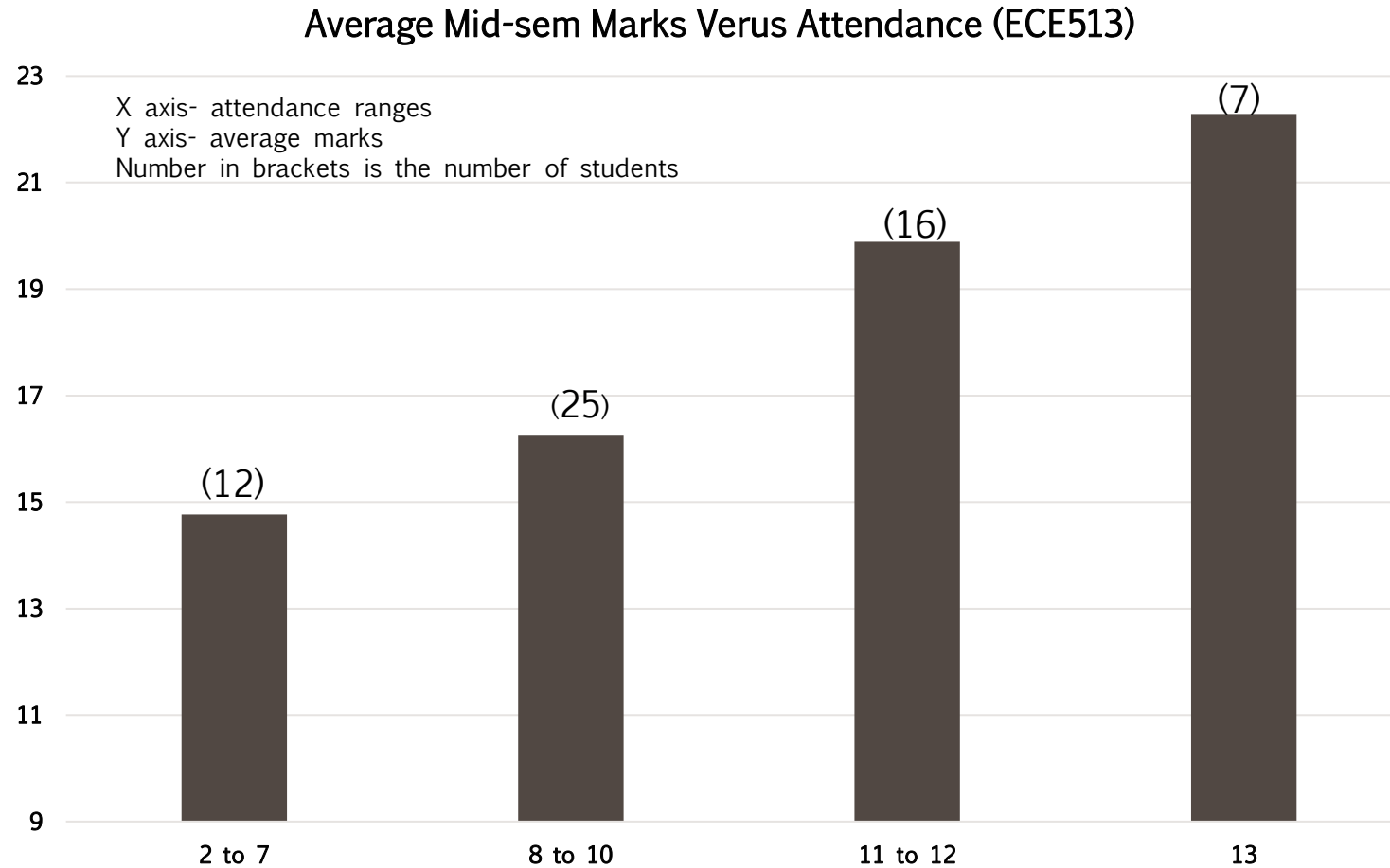
-



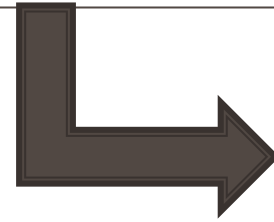
Attendance Vs. Mid-Sem Marks Correlation



Attendance Vs. Mid-Sem Marks Correlation



Static Timing
Analysis

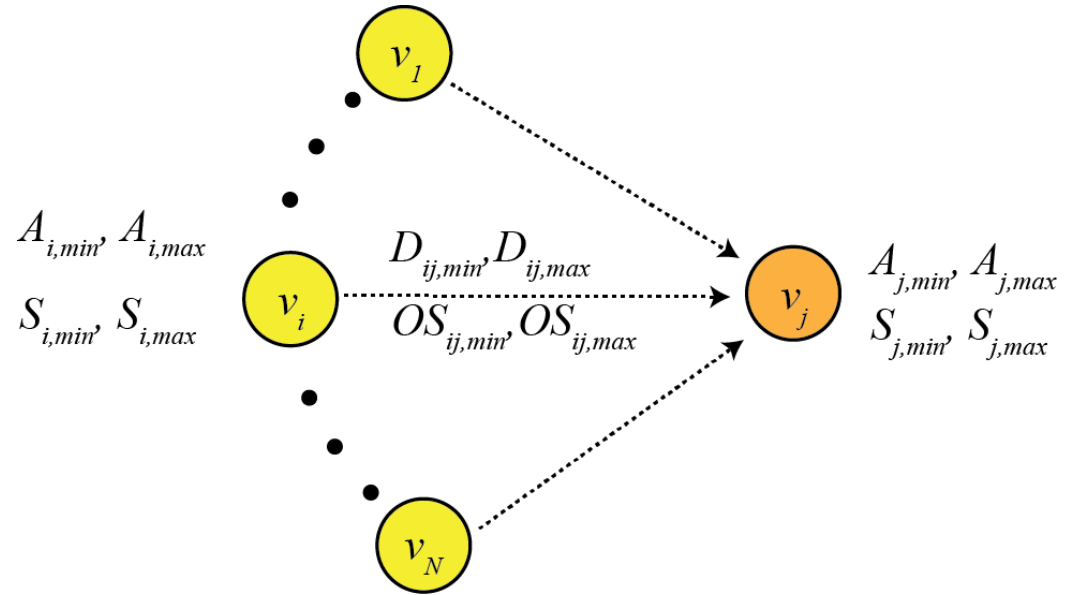


Slew
Propagation

Need for Slew Propagation

- For computing delays for a given stage, the slews at its inputs must be known.
 - An STA tool must also propagate the slews in the timing graph
- The propagated slews can be different through different combinational paths.

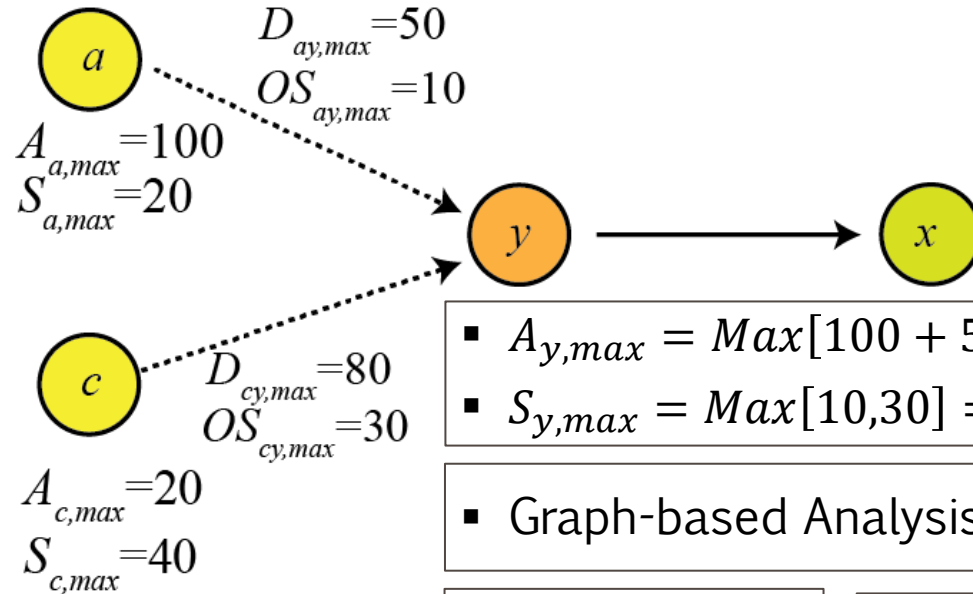
Slew Propagation: Bound on Slew



- $A_{j,min} = \text{Min}[A_{i,min} + D_{ij,min}]$
- $A_{j,max} = \text{Max}[A_{i,max} + D_{ij,max}]$

- $S_{j,min} = \text{Min}[OS_{ij,min}]$
- $S_{j,max} = \text{Max}[OS_{ij,max}]$

Slew Propagation: Example (Maximum Case)



Input Slew S_y	Delay D_{yx}	Output Slew S_x
10	30	10
30	100	20

- $A_{y,max} = \text{Max}[100 + 50, 20 + 80] = 150$
- $S_{y,max} = \text{Max}[10, 30] = 30$

- $A_{x,max} = 150 + 100 = 250$
- $S_{x,max} = 20$

- Graph-based Analysis (GBA): Safe Bound, Not tight (most popular)

- $A_{y,max} = 150$
- $S_{y,max} = 10$

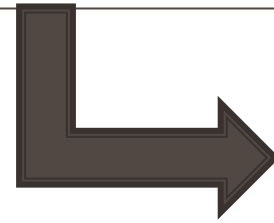
- $A_{x,max} = 180$
- $S_{x,max} = 10$

- $A_{y,max} = 100$
- $S_{y,max} = 30$

- $A_{x,max} = 200$
- $S_{x,max} = 20$

- Path-based Analysis (PBA): Path-specific, computationally difficult

Static Timing
Analysis



Accounting
for Variations

Need to Account for Variations

- Behavior of transistors/circuit elements can differ from the nominal behavior due to process-induced variations and fluctuations in temperature and voltage (PVT variations)
 - Delay and other timing attributes change
 - Can result in timing failure
-
- To tackle variations, different techniques are employed.
 - Differ in accuracy, modeling effort, computational resource requirement, and design effort.

Safety Margins

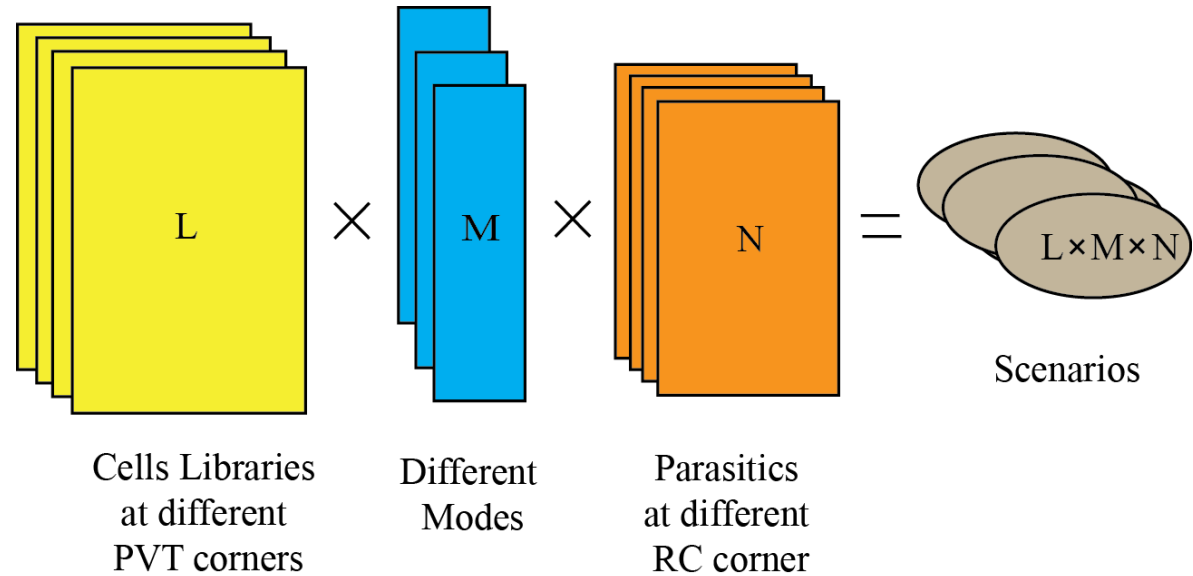
- Easiest technique
 - Can convey margins to an STA tool using appropriate constraints
 - Adjust the required time such that timing requirements become stricter
-
- Large Margin: overly pessimistic (loss in PPA)
 - Small Margin: chance of timing failure, yield loss
 - Need to consider these tradeoffs
-
- Typically employed in early stages of VLSI design flow

Multi-mode Multi-Corner (MMMC) Analysis

- Carry out STA at some discrete set of **scenarios** to account for variations

Scenarios are created by a combination of:

- **PVT corners** for technology libraries: accounts for global variations (worst, best, typical, etc.)
- **Multiple modes**: using SDC files for different modes such functional, test, sleep, turbo, etc.
- **RC corners**: extract multiple SPEF files to account for process-induced variations in interconnects.



- Analyze multiple scenarios simultaneously using MMMC analysis
- Efficiency:
 - Avoiding computation of the dominated scenarios
 - Exploiting parallel processing

On-Chip Variations (OCV) Derate

- Need to account for local variations in the properties of devices and interconnects on the same die

➤ Specify OCV derate factors

- **Effective delay:** obtained by multiplying the nominal delay with the OCV derating factor

Can define different OCV derating factors based on:

- Path bounds (early or late)
- Path type (data or clock)
- Delay type (gate delay or interconnect delay)
- Corners (best, worst, typical, etc.)

Late path derating factor = 1.1

Early path derating factor = 0.9

Setup Analysis:

- Data path and clock launch path: 1.1
- Clock capture path: 0.9

Hold Analysis:

- Data path and clock launch path: 0.9
- Clock capture path: 1.1

Demerit:

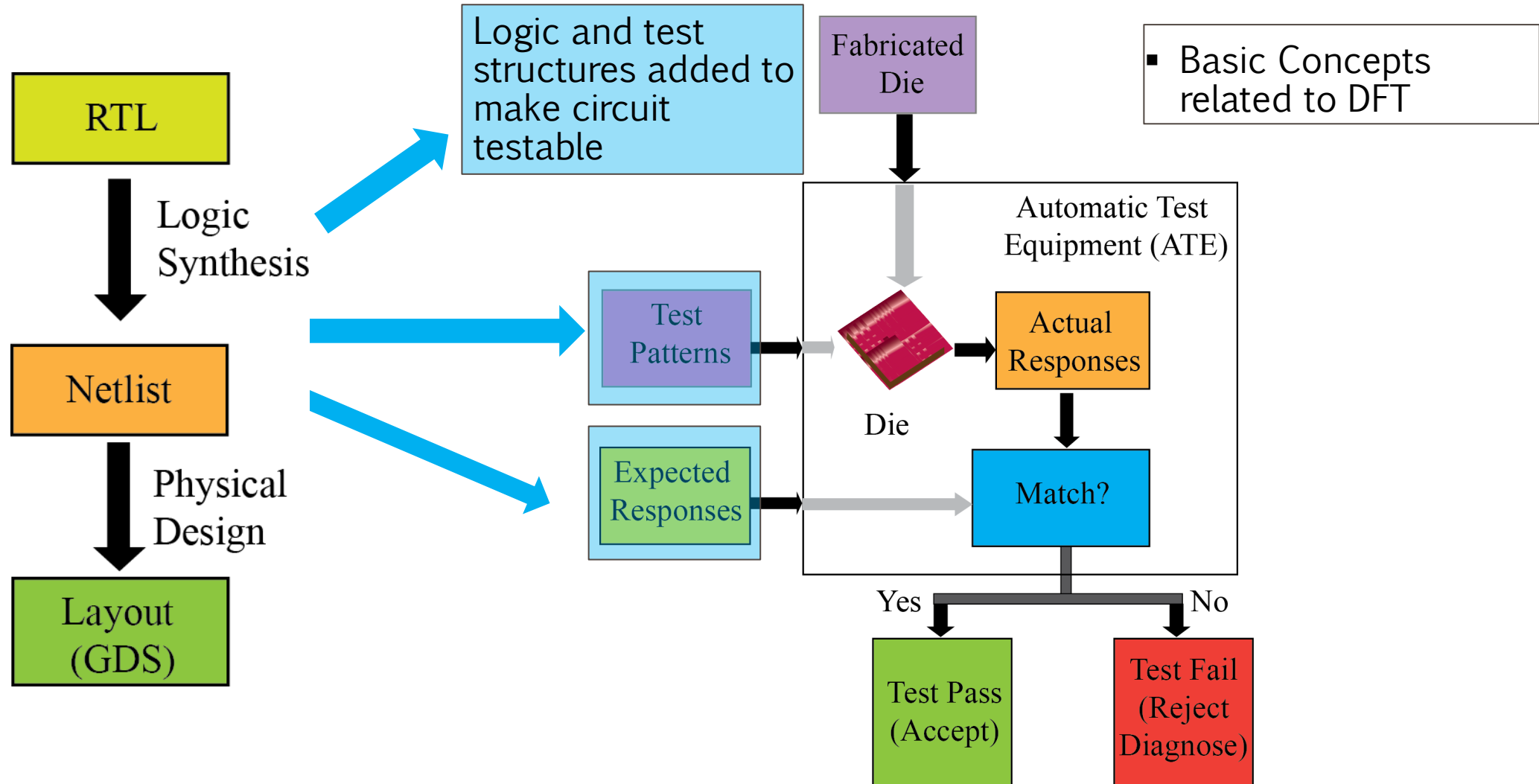
- Assumes perfect positive correlation among timing arcs of same group
- Assumes perfect negative correlation among timing arcs of different group
- Overly pessimistic

VDF: Course Content

Week No.	Topic
1	Introduction: Basic Concepts; Design Styles; Designing vs. Fabrication; Processes involved in taking "Idea" to RTL: Software/Hardware Partitioning, Behavioral Synthesis
2	Overview of "RTL to GDS" Design, Verification and Test Flow; Processes after GDS Tapeout to Final Chip;
3	Design capture: HDL (Verilog), Library and Constraints;
4	RTL Simulation and Synthesis; Static Timing Analysis; Power Analysis;
5	Introduction to Formal Methods: Equivalence Checking, BDD, SAT;
6	Logic Synthesis: Optimization, Technology Mapping;
7	Design for Test (DFT): Fault Models, Scan Insertion, ATPG, BIST
8	Overview of CMOS Fabrication Technology; Basics of Physical Design
9	Floorplanning, Power Planning;
10	Placement: Techniques and Optimization;
11	Clock Tree Synthesis: Distribution, Skew Optimization; Routing: Global and Detailed; ECO
12	Physical Verification: Extraction, LVS, ERC, DRC; Sign-off
13	Summing-up and Paper Presentations

Design For Testability

DFT: In Designing



Structural Testing

Consider a circuit that implements a Boolean function with N inputs.
How to test the fabricated circuit?

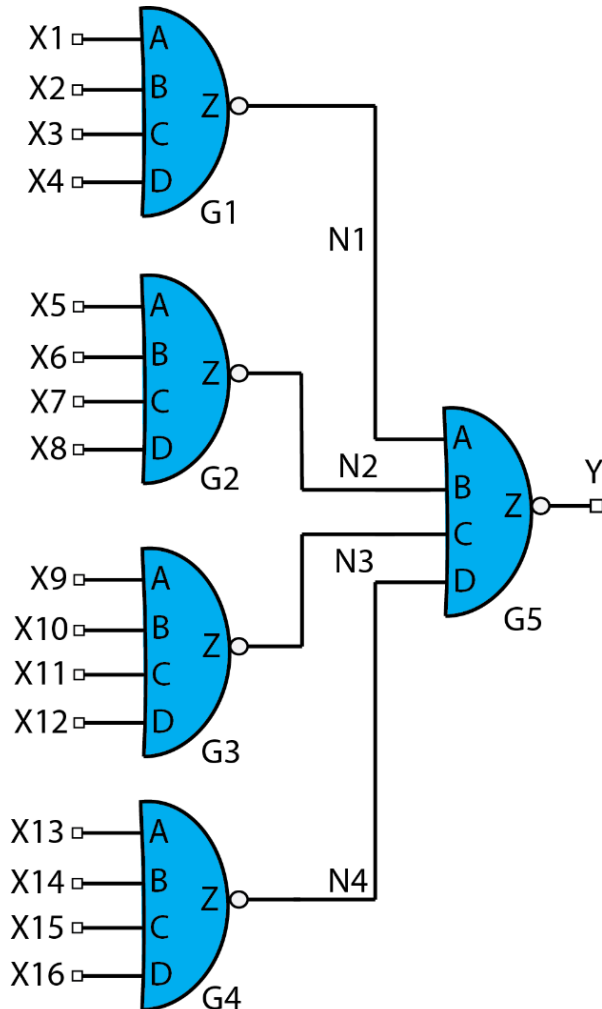
Functional Testing:

- Apply all possible 2^N input combinations and check output
- Becomes infeasible for large N (say 50 or 100)

Structural Testing:

- Test the components that implements a logic function rather than testing the input-output functionality
- The paradigm of structural testing is widely employed
 - It reduces the number of test patterns required for good test quality

Functional Testing vs. Structural Testing



Functional Testing

- Will require $2^{16} = 65536$ input combinations

Structural Testing

- Test individual components of the circuit ($G1$, $G2$, $G3$, $G4$, and $G5$)
- Each gate will require $2^4 = 16$ input combinations (total 80)

Assumptions

- Can observe the output pins for all the components
- Can write any value at the input pins of all the components
- There could be a problem in the integration of components (need to tackle that also).

DFT techniques tries to make structural testing more effective by ensuring the above assumptions.

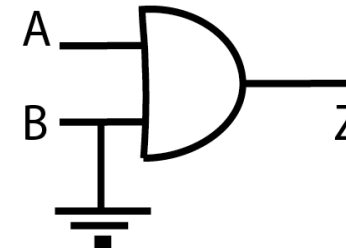
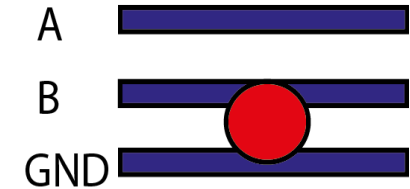
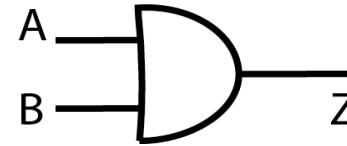
Fault Models

Fault Model

- Represents a defect using a logical or electrical model

Why do we use a Fault Model?

- Allows us to analyze the impact of a defect using logic or circuit analysis techniques
- Allows deriving test patterns algorithmically for detecting a given fault
 - Quantitative assessment of testing effectiveness using fault coverage
- Transforms the problem of defect detection to the problem of fault detection



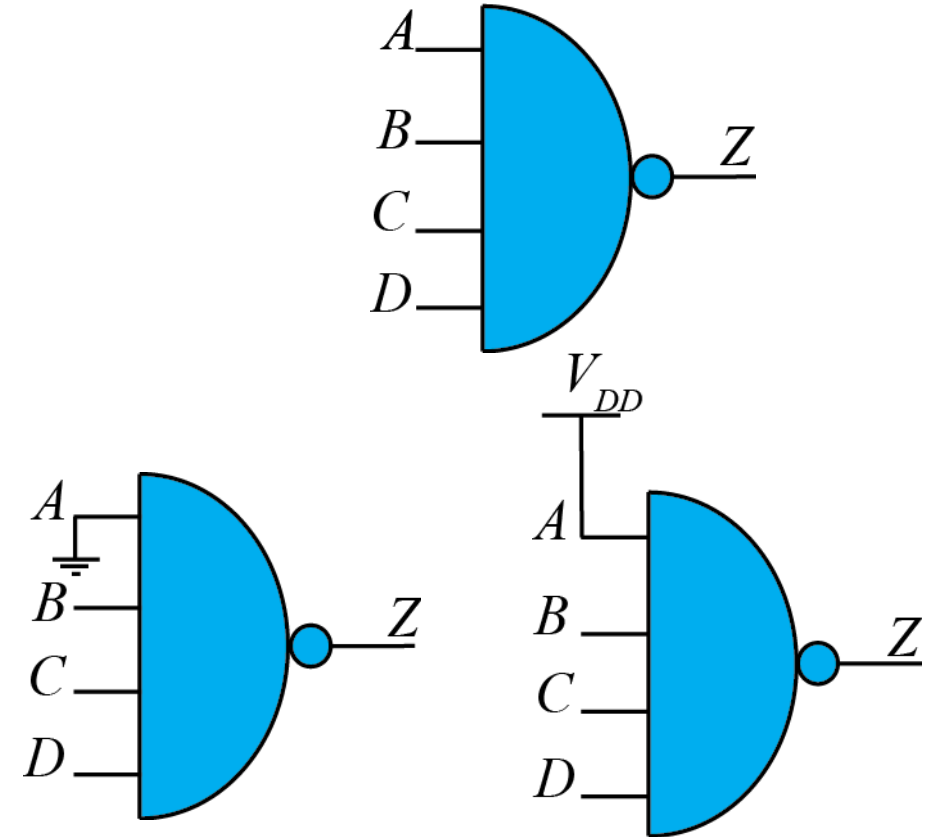
Stuck-at Fault Models

Stuck-at Fault Model

- Assumes that defects cause the signal to be permanently stuck at a constant logic
- Transforms defects to a logical fault model
- Two types:
 - Stuck at logic “0”: stuck-at-0 or SA0 fault
 - Stuck at logic “1”: stuck-at-1 or SA1 fault

Single Stuck-at Fault Model

- Assumes that there is only one fault active at a time
- Reduces the complexity of test pattern generation significantly



- A Stuck-at 0
- A/0

- A Stuck-at 1
- A/1

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.